Inventory Stock Management System

A Full-Stack Python + MongoDB Application

Subject:	Database Management Systems
Technology:	Python, Flask, MongoDB, PyMongo, HTML/CSS/JS
Database:	MongoDB (NoSQL Document Store)
Date:	October 19, 2026

College Assignment - Project Report

Table of Contents

1. Introduction & Problem Statement
2. Real-World Applications
3. Objectives & Scope
4. Technology Stack
5. System Architecture
6. Database Design (MongoDB)
7. Indexing Strategy
8. Core Features & Implementation
9. Aggregation Pipelines (Reports)
10. Atomic Operations (Stock Management)
11. REST API Design
12. Frontend Design
13. Project File Structure
14. How to Run the Project
15. Future Enhancements
16. Conclusion
17. References

1. Introduction & Problem Statement

Inventory management is a critical aspect of any retail business, warehouse, or institutional store. It involves tracking the quantity, location, and status of goods across the supply chain. In many small retail shops, college stores, and local businesses, inventory is still managed using paper registers or spreadsheets, leading to several operational challenges.

Problems with Manual Inventory Management

- Stockouts: Running out of items without any warning system, leading to lost sales and customer dissatisfaction.
- Overstocking: Excessive inventory ties up working capital and increases storage costs.
- No Audit Trail: It is impossible to trace who added or removed stock, making accountability difficult.
- Human Errors: Manual data entry is prone to mistakes in quantity, pricing, and categorization.
- Slow Reporting: Generating reports like 'which items are low?' or 'what is the total inventory value?' requires hours.
- No Real-time Visibility: Managers cannot see current stock levels without physically counting items.

Our Solution

This project - the Inventory Stock Management System - addresses all of the above problems by providing a digital inventory management solution built with Python and MongoDB. It offers both a Command-Line Interface (CLI) for quick terminal-based operations and a modern Web Dashboard for visual, browser-based management. The system supports full CRUD operations on products and suppliers, atomic stock movements, real-time low-stock alerts, and powerful aggregation-based analytics reports.

2. Real-World Applications

This system is not just an academic exercise - it models real-world inventory management patterns used by businesses of all sizes. Below are practical scenarios where this system can be directly deployed or adapted:

Retail Shops & Kirana Stores

A local shop owner can track all products, set reorder levels for automatic low-stock alerts, record daily sales (stock OUT), and purchases from suppliers (stock IN). The CSV export feature helps with tax filing and bookkeeping.

College & University Stores

Campus bookstores, lab supply rooms, and hostel canteens can manage stationery, chemicals, or food items. Multiple staff members can use the web interface simultaneously.

Warehouse Management

Warehouses receiving goods from multiple suppliers can track each supplier's products, monitor total inventory value, and generate monthly movement reports for logistics planning.

Pharmacy & Medical Stores

Medicines with critical stock levels can be monitored using the reorder_level feature. The audit trail (transactions collection) ensures regulatory compliance.

E-commerce Order Fulfillment

The atomic stock-out operation prevents overselling - if two orders arrive simultaneously for the last item, MongoDB's atomic guard ensures only one succeeds, preventing negative stock.

Restaurant & Cafe Inventory

Track ingredients, beverages, and supplies. Monthly movement reports show consumption patterns to optimize purchasing decisions.

3. Objectives & Scope

Primary Objectives

- Demonstrate proficiency in MongoDB - CRUD operations, indexing, text search, and aggregation pipelines.
- Build a functional inventory system with Product Management, Supplier Management, Stock Operations, and Reporting.
- Implement atomic database operations to prevent race conditions in concurrent stock updates.
- Provide both a CLI and a Web-based interface for maximum flexibility.
- Generate 5 analytical reports using MongoDB's Aggregation Framework.
- Export data to CSV format for external use and backup.

Scope

Feature	In Scope	Out of Scope
Product CRUD	Yes	
Supplier CRUD	Yes	
Stock IN/OUT (Atomic)	Yes	
5 Aggregation Reports	Yes	
Text Search + Filters	Yes	
CSV Export	Yes	
Web Dashboard	Yes	
User Authentication		Future Enhancement
Barcode/QR Scanning		Future Enhancement
Cloud Deployment		Future Enhancement

4. Technology Stack

Layer	Technology	Purpose
Language	Python 3.x	Core application logic
Database	MongoDB 7.x	NoSQL document store for all data
DB Driver	PyMongo 4.7	Python driver for MongoDB operations
Web Framework	Flask 3.x	Lightweight REST API server
CORS	Flask-CORS	Cross-Origin Resource Sharing for API
Frontend	HTML5+CSS3+JS	Modern single-page web dashboard
Config	python-dotenv	Environment variable management
Display	tabulate	CLI table formatting
Seed Data	Faker	Realistic test data generation
Testing	Pytest	Unit and integration testing

Why MongoDB (NoSQL) over SQL?

Aspect	MongoDB (NoSQL)	MySQL/PostgreSQL (SQL)
Data Model	Flexible JSON documents	Rigid table schemas
Schema Changes	No migrations needed	ALTER TABLE required
Nested Data	Tags array, embedded docs	Separate join tables
Aggregation	Pipeline stages (flexible)	GROUP BY (rigid)
Scaling	Horizontal (sharding)	Vertical (bigger server)
Setup	No schema definition	CREATE TABLE + types
Best For	Evolving applications	Fixed-schema, ACID-heavy

5. System Architecture

Three-Tier Architecture

The application follows a clean three-tier architecture pattern, separating concerns across distinct layers. Each layer communicates only with its adjacent layer, promoting modularity and testability.

Tier	Components	Responsibility
Presentation	frontend/ + cli/	User interface (web + CLI)
Application	server.py + services/	Business logic, API routing
Data	db/ + MongoDB	Database connection, indexes, storage

Request Flow (Adding a Product)

> 1. Browser sends HTTP POST /api/products with JSON body.
> 2. Flask (server.py) receives the request and parses JSON data.
> 3. Flask calls the service function: add_product().
> 4. Service uses PyMongo to execute insert_one() on MongoDB.
> 5. MongoDB stores the document and returns the ObjectId.
> 6. Service returns a success/failure dict to Flask.
> 7. Flask serializes the response to JSON and returns to browser.
> 8. Browser JS shows a success toast and refreshes the table.

Singleton Connection Pattern

The database connection uses the Singleton pattern (db/connection.py). PyMongo's MongoClient maintains an internal connection pool, so creating one per process is the recommended approach. The module-level variable _client is initialized on the first call to get_client() and reused for all subsequent calls.

db/connection.py - Singleton MongoClient

```
# Singleton pattern - one MongoClient per process
_client = None

def get_client():
    global _client
    if _client is None:
        _client = MongoClient(MONGO_URI, timeout=3000)
        _client.admin.command("ping")  # verify connection
    return _client
```

6. Database Design (MongoDB)

The database 'inventory_db' contains three collections. Unlike SQL databases, MongoDB does not require predefined schemas - documents can have varying fields. However, our application enforces consistent structure through builder functions in models/schemas.py.

Collection 1: products

Field	Type	Required	Description
_id	ObjectId	Auto	Unique document ID (auto-generated)
sku	String	Yes	Stock Keeping Unit - unique code
name	String	Yes	Product display name
category	String	Yes	Product category
price	Float	Yes	Unit price in INR
quantity	Integer	Yes	Current stock count
reorder_level	Integer	No	Low-stock alert threshold (def: 10)
supplier_id	ObjectId	No	Reference to supplier document
tags	Array	No	Searchable tags list
created_at	DateTime	Auto	Creation timestamp (UTC)
updated_at	DateTime	Auto	Last modification timestamp

Collection 2: suppliers

Field	Type	Required	Description
_id	ObjectId	Auto	Unique document ID
name	String	Yes	Supplier company name
email	String	Yes	Contact email (unique)
phone	String	No	Contact phone number
address	String	No	Physical address
created_at	DateTime	Auto	Registration timestamp

Collection 3: transactions

Every stock movement (IN or OUT) is recorded as a transaction document. This collection serves as an audit trail and is the data source for movement reports.

Field	Type	Required	Description
_id	ObjectId	Auto	Unique document ID
product_id	ObjectId	Yes	Reference to product that moved
type	String	Yes	Movement type: 'IN' or 'OUT'
quantity	Integer	Yes	Number of units moved
note	String	No	Optional note (PO number, etc.)
timestamp	DateTime	Auto	When the movement occurred

Entity Relationships

- products.supplier_id references suppliers._id (manual join via $lookup).
- transactions.product_id references products._id (for stock history).
- Supplier deletion is blocked if products still reference that supplier.

7. Indexing Strategy

Indexes in MongoDB work like indexes in a book - they allow the database engine to locate documents without scanning every document in the collection (called a collection scan). Without indexes, MongoDB must perform a full scan for every query, which becomes slow as data grows.

Collection	Field(s)	Index Type	Purpose
products	sku	Unique	Prevents duplicate SKUs
products	category	Regular	Fast category filtering
products	name	Text	Full-text search
suppliers	email	Unique	Prevents duplicate emails
transactions	(product_id, timestamp)	Compound	Fast history queries

db/indexes.py - Index Definitions

```
# Indexes are created at startup (idempotent)
products.create_index("sku", unique=True)
products.create_index("category")
products.create_index([("name", "text")])
suppliers.create_index("email", unique=True)
transactions.create_index(
    [("product_id", ASCENDING), ("timestamp", DESCENDING)]
)
```

Unique indexes also serve as constraints - attempting to insert a document with a duplicate SKU or email raises a DuplicateKeyError, which we catch and return a friendly error message to the user.

8. Core Features & Implementation

8.1 Product Management (CRUD)

The product service (services/product_service.py) implements all four CRUD operations:

Operation	PyMongo Method	SQL Equivalent
Create	insert_one(doc)	INSERT INTO products VALUES(...)
Read All	find({}).sort().skip().limit()	SELECT * ... ORDER BY ... LIMIT
Read One	find_one({'sku': sku})	SELECT * WHERE sku='X' LIMIT 1
Update	update_one(filter, {'$set': ...})	UPDATE products SET ... WHERE
Delete	delete_one({'sku': sku})	DELETE FROM products WHERE
Count	count_documents({})	SELECT COUNT(*)
Distinct	distinct('category')	SELECT DISTINCT category

8.2 Text Search

MongoDB's $text operator uses the text index on the 'name' field for full-text search. It supports stemming (e.g., 'laptop' matches 'laptops') and relevance scoring. If text search returns no results, a regex fallback performs partial matching.

product_service.py - Search Implementation

```
# Primary: Full-text search (fast, uses text index)
col.find(
    {"$text": {"$search": "wireless keyboard"}},
    {"score": {"$meta": "textScore"}}
).sort([("score", {"$meta": "textScore"})])

# Fallback: Regex search (slower, partial matches)
col.find({"name": {"$regex": "key", "$options": "i"}})
```

8.3 Supplier Management

Suppliers support full CRUD operations. A critical feature is referential integrity enforcement: a supplier cannot be deleted if any products reference their supplier_id. This check is performed manually in the service layer since MongoDB does not enforce foreign keys.

supplier_service.py - Referential Integrity Check

```
# Before deleting a supplier, check for linked products
linked = products_col.count_documents({"supplier_id": oid})
if linked > 0:
    return {"error": f"{linked} products still linked"}
```

8.4 Pagination

Product listing uses server-side pagination with MongoDB's skip() and limit() methods.

product_service.py - Pagination with skip/limit

```
skip_count = (page - 1) * PAGE_SIZE
products = list(
    col.find({}).sort("sku", 1).skip(skip_count).limit(PAGE_SIZE)
)
total = col.count_documents({})
total_pages = -(-total // PAGE_SIZE)  # ceiling division
```

9. Aggregation Pipelines (Reports)

MongoDB's Aggregation Framework is one of its most powerful features. A pipeline is an ordered array of stages - each stage transforms its input documents and passes the result to the next, similar to Unix pipes. The system implements 5 report pipelines:

Pipeline Stages Used

Stage	SQL Equivalent	Description
$match	WHERE	Filters documents
$group	GROUP BY	Groups and aggregates (SUM, COUNT)
$sort	ORDER BY	Sorts results
$limit	LIMIT	Restricts result count
$lookup	LEFT JOIN	Joins with another collection
$unwind	Flatten	Deconstructs array into docs
$project	SELECT / AS	Reshapes output documents
$expr	Compare cols	Compares two fields in same doc

Report 1: Low Stock Alert

Finds products where quantity <= reorder_level using $expr to compare two fields.

```
col.find(
    {"$expr": {"$lte": ["$quantity", "$reorder_level"]}}
).sort("quantity", 1)
```

Report 2: Category Stock Value

Groups by category, counts products, sums quantities, calculates total value (price x qty).

```
{"$group": {
    "_id": "$category",
    "total_products": {"$sum": 1},
    "total_value": {"$sum": {"$multiply": ["$price", "$quantity"]}}
}}
{"$sort": {"total_value": -1}}
```

Report 3: Supplier Summary

Groups products by supplier_id, then uses $lookup to JOIN with suppliers collection.

```
{"$group": {"_id": "$supplier_id", ...}}
{"$lookup": {
    "from": "suppliers",
    "localField": "_id",
    "foreignField": "_id",
    "as": "supplier_info"
}}
{"$unwind": "$supplier_info"}
```

Report 4: Monthly Movement

Groups transactions by year, month, and type using date extraction operators.

```
{"$group": {
    "_id": {
        "year": {"$year": "$timestamp"},
        "month": {"$month": "$timestamp"},
        "type": "$type"
    },
    "total_quantity": {"$sum": "$quantity"}
}}
```

Report 5: Top 5 Sold Products

A 7-stage pipeline: $match -> $group -> $sort -> $limit -> $lookup -> $unwind -> $project.

```
{"$match": {"type": "OUT"}}
{"$group": {"_id": "$product_id",
            "total_sold": {"$sum": "$quantity"}}}
{"$sort": {"total_sold": -1}}
{"$limit": 5}
{"$lookup": {"from": "products", ...}}
```

10. Atomic Operations (Stock Management)

Stock management is the most technically challenging part of the system. The key problem is ensuring data consistency when multiple users update stock simultaneously.

The Race Condition Problem

Consider the naive approach to Stock OUT: 1) Read the product (quantity = 5). 2) Check if quantity >= requested (5 >= 3? Yes). 3) Update: set quantity = 5 - 3 = 2. If two users execute this simultaneously, both read quantity = 5 at step 1, both pass the check, and both subtract 3 - resulting in quantity = -1, which is impossible!

The Atomic Guard Solution

MongoDB solves this by combining the check and update into a SINGLE atomic operation. The guard condition is embedded inside the query filter:

stock_service.py - Atomic Guard Pattern

```
# ATOMIC stock-out: check + update in ONE operation
result = col.update_one(
    {
        "_id": product_id,
        "quantity": {"$gte": requested_qty}   # GUARD
    },
    {
        "$inc": {"quantity": -requested_qty},  # decrement
        "$set": {"updated_at": now}
    }
)

# If matched_count == 0, guard failed (insufficient stock)
if result.matched_count == 0:
    return {"error": "Insufficient stock"}
```

Why this works: MongoDB guarantees that update_one() is atomic at the document level. No other operation can modify the document between the filter evaluation and the update. If quantity was already reduced by another concurrent request, the filter won't match, and matched_count will be 0 - preventing negative stock.

Stock IN - Using $inc Operator

stock_service.py - Atomic Stock IN

```
# $inc atomically adds to the current value
col.update_one(
    {"_id": product_id},
    {
        "$inc": {"quantity": 20},      # quantity += 20
        "$set": {"updated_at": now}    # update timestamp
    }
)
```

11. REST API Design

The Flask server (server.py) exposes a RESTful API following standard HTTP conventions. All endpoints accept/return JSON. The API layer only handles HTTP routing - all business logic is delegated to the service layer.

Product Endpoints

Method	Endpoint	Description
GET	/api/products?page=1	List products (paginated)
POST	/api/products	Add a new product
GET	/api/products/<sku>	Get product by SKU
PUT	/api/products/<sku>	Update product fields
DELETE	/api/products/<sku>	Delete a product
GET	/api/categories	Get distinct categories

Supplier Endpoints

Method	Endpoint	Description
GET	/api/suppliers	List all suppliers
POST	/api/suppliers	Add a new supplier
GET	/api/suppliers/<id>	Get supplier by ID
PUT	/api/suppliers/<id>	Update supplier fields
DELETE	/api/suppliers/<id>	Delete a supplier

Stock & Report Endpoints

Method	Endpoint	Description
POST	/api/stock/in	Record Stock IN operation
POST	/api/stock/out	Record Stock OUT operation
GET	/api/stock/history/<sku>	Transaction history
GET	/api/dashboard	Summary stats
GET	/api/reports/low-stock	Low stock report
GET	/api/reports/category-value	Category value report
GET	/api/reports/supplier-summary	Supplier summary

GET	/api/reports/monthly-movement	Monthly movements
GET	/api/reports/top-sold	Top 5 sold products
POST	/api/export/csv	Export products to CSV

12. Frontend Design

The web frontend is a Single-Page Application (SPA) built with vanilla HTML, CSS, and JavaScript - no frameworks like React or Angular are used. It communicates with the Flask API via fetch() requests.

Design Features

- Dark theme with glassmorphism effects for a premium look.
- Sidebar navigation with smooth page transitions.
- Real-time search with debouncing (400ms delay).
- Modal dialogs for add/edit/delete operations.
- Toast notifications for success/error feedback.
- Responsive layout - works on desktop and tablet.
- Color-coded stock badges (In Stock, Low Stock, Out of Stock).
- Paginated product tables with category filtering.

Frontend Pages

Page	Features
Dashboard	6 stat cards, low stock alerts, top sellers
Products	CRUD table, search, filter, pagination
Suppliers	CRUD table with integrity protection
Stock Ops	Side-by-side IN/OUT forms, history
Reports	5 analytics reports in card layout

13. Project File Structure

```
inventory-management/
|
|-- server.py               # Flask REST API server
|-- main.py                 # CLI entry point
|-- config.py               # Loads .env configuration
|-- .env                    # Environment variables
|-- requirements.txt        # Python dependencies
|
|-- frontend/               # Web UI (SPA)
|   |-- index.html          # HTML structure
|   |-- style.css           # Dark theme stylesheet
|   |-- app.js              # Client-side JavaScript
|
|-- db/                     # Database layer
|   |-- connection.py       # Singleton MongoClient
|   |-- indexes.py          # Index definitions
|
|-- models/
|   |-- schemas.py          # Dataclass schemas + builders
|
|-- services/               # Business logic layer
|   |-- product_service.py  # Product CRUD + search
|   |-- supplier_service.py # Supplier CRUD
|   |-- stock_service.py    # Atomic stock operations
|   |-- report_service.py   # 5 aggregation reports
|
|-- utils/
|   |-- exporter.py         # CSV export utility
|   |-- validators.py       # Input validation
|
|-- cli/                    # CLI interface layer
|-- tests/                  # Pytest test suite
|-- scripts/                # Seed data script
|-- docs/                   # Documentation
```

14. How to Run the Project

Prerequisites

- Python 3.10+ installed
- MongoDB 6.0+ installed and running
- pip (Python package manager)

Installation Steps

Setup & Run Commands

```
# 1. Clone the repository
git clone <repository-url>
cd inventory-management

# 2. Create and activate virtual environment
python -m venv venv
source venv/bin/activate      # macOS/Linux

# 3. Install dependencies
pip install -r requirements.txt
pip install flask flask-cors

# 4. Configure environment
cp .env.example .env

# 5. Start MongoDB
brew services start mongodb-community   # macOS

# 6. Run the web application
python server.py
# Open http://localhost:5000

# OR run the CLI version
python main.py
```

15. Future Enhancements

- User Authentication: JWT-based login with role-based access (admin, staff).
- Barcode/QR Scanning: Integrate scanner for faster stock operations.
- Purchase Order Management: Generate and track POs with suppliers.
- Multi-location Support: Track inventory across warehouses.
- Email Notifications: Auto alerts when stock falls below reorder level.
- Dashboard Charts: Interactive charts using Chart.js for visuals.
- Cloud Deployment: Deploy on MongoDB Atlas + Railway for remote access.
- Mobile App: Companion app using React Native.
- Batch Import: Upload CSV/Excel to bulk-add products.
- Audit Logging: Track all user actions (who changed what, when).

16. Conclusion

The Inventory Stock Management System successfully demonstrates the practical application of MongoDB and PyMongo in building a real-world database-driven application. The project covers all essential database concepts including:

- CRUD operations using insert_one, find, update_one, and delete_one.
- Indexing with unique, text, compound, and regular indexes.
- Aggregation Pipelines with 7 different stages across 5 reports.
- Atomic Operations to prevent race conditions in stock updates.
- Schema Design with document references (simulating foreign keys).
- Full-Text Search with text indexes and relevance scoring.
- Pagination with skip/limit for efficient data retrieval.

By providing both a CLI and a modern web interface, the system demonstrates how a service-oriented architecture allows multiple frontends to share the same business logic and database layer. The project is ready for production use in small retail environments.

17. References

[1] MongoDB Official Documentation - https://www.mongodb.com/docs/manual/

[2] PyMongo Documentation - https://pymongo.readthedocs.io/en/stable/

[3] Flask Documentation - https://flask.palletsprojects.com/

[4] MongoDB Aggregation Pipeline - mongodb.com/docs/manual/core/aggregation-pipeline/

[5] MongoDB Indexes - https://www.mongodb.com/docs/manual/indexes/

[6] MongoDB Atomic Operations - mongodb.com/docs/manual/core/write-operations-atomicity/

[7] Python Official Documentation - https://docs.python.org/3/

[8] MDN Web Docs (HTML/CSS/JS) - https://developer.mozilla.org/